MCA Lab Assignments Compilation

Progressive Education Society's Modern College of Engineering | Python Implementations

Part 1: Basic Programming & Data Structures

1. Write a Python/R program to create three vectors a,b,c with 3 integers. Combine the three vectors to become a 3×3 matrix where each column represents a vector. Print the content of the matrix.

```
import numpy as np # Create three vectors a = np.array([1, 2, 3]) b = np.array([4, 5, 6]) c = np.array([7, 8, 9]) # Combine them column-wise matrix = np.column_stack((a, b, c)) print("3x3 Matrix:\n", matrix)
```

2. Write a Python/R program to create a list containing a vector, a matrix and a list and give names to the elements in the list. Access the first and second element of the list.

```
import numpy as np # Create a dictionary (Python's equivalent of a named list) named_list = { 'my_vector': np.array([10, 20, 30]), 'my_matrix': np.array([[1, 2], [3, 4]]), 'my_list': [100, 200, 300] } # Access the first and second elements first_element = list(named_list.values())[0] second_element = list(named_list.values())[1] print("First element (Vector):\n", first_element) print("Second element (Matrix):\n", second_element)
```

3. Write a Python/R program to create an array with three columns, three rows, and two "tables", taking two vectors as input to the array. Print the array.

```
import numpy as np # Create two input vectors v1 = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9]) v2 = np.array([10, 11, 12, 13, 14, 15, 16, 17, 18]) # Combine and reshape into (2 tables, 3 rows, 3 columns) array_3d = np.concatenate((v1, v2)).reshape(2, 3, 3) print("3D Array (2 Tables, 3 Rows, 3 Columns):\n", array_3d)
```

4. Write a Python/R program to create a data frame from four given vectors (name, score, attempts, qualify).

```
import pandas as pd # Create the data dictionary data = { 'name': ['Anastasia', 'Dima', 'Katherine', 'James', 'Emily', 'Michael', 'Matthew', 'Laura', 'Kevin', 'Jonas'], 'score': [12.5, 9, 16.5, 12, 9, 20, 14.5, 13.5, 8, 19], 'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1], 'qualify': ['yes', 'no', 'yes', 'no', 'no', 'yes', 'yes', 'no', 'no', 'yes'] } # Convert to DataFrame df = pd.DataFrame(data) print(df)
```

5. Write a Python/R program to create a factor corresponding to height of women data set, which contains height and weights for a sample of women.

```
import pandas as pd # Sample dataset women_df = pd.DataFrame({ 'Height': [58, 59, 60, 61, 62, 63, 64], 'Weight': [115, 117, 120, 123, 126, 129, 132] }) # Create a factor (categorical variable) based on height bins women_df['Height_Factor'] = pd.cut(women_df['Height'], bins=[0, 60, 65], labels=['Short', 'Average']) print(women_df)
```

6. Write Python/R program to Print the Fibonacci Sequence.

```
def print_fibonacci(n_terms): a, b = 0, 1 sequence = [] for _ in range(n_terms): sequence.append(a) a, b = b, a + b return sequence print("Fibonacci Sequence (first 10 terms):", print_fibonacci(10))
```

7. Write Python/R program to import and export data from csv, excel, txt file.

```
import pandas as pd # Create a sample dataframe df = pd.DataFrame({'ID': [1, 2], 'Name': ['A', 'B']}) # EXPORT df.to_csv('data.csv', index=False) df.to_excel('data.xlsx', index=False) df.to_csv('data.txt', sep='\t', index=False) print("Data exported successfully.") # IMPORT df_imported = pd.read_csv('data.csv') print("\nImported DataFrame from CSV:\n", df_imported)
```

8. Write a Python/R program to find the sum of digits of a number reducing it to one digit using repeat loop.

```
def single_digit_sum(n): while n > 9: # Sum the digits of the current number n = sum(int(digit) for digit in str(n)) return n number = 9875 print(f"Single digit sum of {number} is:", single_digit_sum(number))
```

Part 2: Machine Learning & Data Analytics

LA1 (Q2). Use the Iris dataset and train a Logistic Regression model to classify whether a flower is of species Iris-setosa or not. Evaluate model accuracy, precision, and recall.

```
from sklearn.datasets import load_iris from sklearn.model_selection import train_test_split from sklearn.linear_model import LogisticRegression from sklearn.metrics import accuracy_score, precision_score, recall_score # Load data and set target (1 if Setosa, 0 otherwise) iris = load_iris() X = iris.data y = (iris.target == 0).astype(int) X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42) # Train and predict model = LogisticRegression() model.fit(X_train, y_train) y_pred = model.predict(X_test) print("Accuracy:", accuracy_score(y_test, y_pred)) print("Precision:", precision_score(y_test, y_pred)) print("Recall:", recall_score(y_test, y_pred)) # Corrected Syntax
```

LA2 (Q3). Load the Titanic dataset, perform basic preprocessing (handle missing values, encode categorical variables), and train a Decision Tree classifier. Plot the confusion matrix.

```
import pandas as pd import matplotlib.pyplot as plt from sklearn.model_selection import train_test_split from sklearn.tree import DecisionTreeClassifier from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay # 1. Load the actual Titanic dataset from local CSV titanic = pd.read_csv('titanic.csv') # 2. Basic Preprocessing titanic = titanic[['Survived', 'Pclass', 'Sex', 'Age']] titanic['Age'] = titanic['Age'].fillna(titanic['Age'].median()) titanic['Sex'] = titanic['Sex'].map({'male': 0, 'female': 1}) X = titanic[['Pclass', 'Sex', 'Age']] y = titanic['Survived'] # 3. Train Model X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42) dt = DecisionTreeClassifier(random_state=42) dt.fit(X_train, y_train) # 4. Plot Confusion Matrix cm = confusion_matrix(y_test, dt.predict(X_test)) disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Died', 'Survived']) disp.plot(cmap='Blues') plt.title("Titanic Confusion Matrix") plt.show()
```

LA3 (Q4). Use the Naive Bayes classifier to perform sentiment analysis on a dataset of movie reviews, classifying them as either positive or negative based on the review text.

```
from sklearn.feature_extraction.text import CountVectorizer from sklearn.naive_bayes import MultinomialNB # Mock dataset reviews = ["I loved this movie", "Terrible film, hated it", "Great acting and plot", "Worst movie ever"] labels = [1, 0, 1, 0] # 1: Positive, 0: Negative # Vectorize text vectorizer = CountVectorizer() X = vectorizer.fit_transform(reviews) # Train model nb = MultinomialNB() nb.fit(X, labels) # Test model test_reviews = ["The movie was great", "I hated the plot"] X_test = vectorizer.transform(test_reviews) print("Predictions (1=Pos, 0=Neg):", nb.predict(X_test))
```

LA4 (Q6). A company wants to predict the salary of employees based on their years of experience. Use simple linear regression to model the relationship between salary and experience.

```
import numpy as np import matplotlib.pyplot as plt from sklearn.linear_model import LinearRegression # Generate mock data np.random.seed(42) experience = np.random.uniform(1, 20, 100).reshape(-1, 1) salary = 30000 + (experience * 5000) + np.random.normal(0, 10000, (100, 1)) # Train model reg = LinearRegression() reg.fit(experience, salary) # Plotting with explicit parameters for clarity plt.figure(figsize=(8, 6)) plt.scatter(experience, salary, color='blue', label='Actual Data') plt.plot(experience, reg.predict(experience), color='red', linewidth=2, label='Regression Line') plt.title("Employee Salary vs. Years of Experience") plt.xlabel("Years of Experience") plt.ylabel("Salary (USD)") plt.grid(True, linestyle='--', alpha=0.6) plt.legend() plt.show()
```

LA5 (Q7). Use a dataset with features like square footage vs. house price. Implement a simple linear regression model to predict house prices. Plot the regression line and calculate MSE and R^2 score.

```
import numpy as np import matplotlib.pyplot as plt from sklearn.linear_model import LinearRegression from sklearn.metrics import mean_squared_error, r2_score # Generate mock data sq_foot = np.random.uniform(500, 5000, 100).reshape(-1, 1) price = 50000 + (sq_foot * 150) + np.random.normal(0, 50000, (100, 1)) # Train model reg = LinearRegression() reg.fit(sq_foot, price) y_pred = reg.predict(sq_foot) print("MSE:", mean_squared_error(price, y_pred)) print("R² Score:", r2_score(price, y_pred)) # Plotting with explicit parameters for clarity plt.figure(figsize=(8, 6)) plt.scatter(sq_foot, price, color='green', label='Actual House Prices') plt.plot(sq_foot, y_pred, color='orange', linewidth=2, label='Regression Line') plt.title("House Price Prediction based on Square Footage") plt.xlabel("Square Footage") plt.ylabel("Price (USD)") plt.grid(True, linestyle='--', alpha=0.6) plt.legend() plt.show()
```

LA6 (Q8). Load the tissue_gene_expression dataset. Run a k-means clustering on the data with K=7. Make a table comparing the identified clusters to the actual tissue types.

```
import pandas as pd import warnings from sklearn.cluster import KMeans from sklearn.datasets import load_wine # Suppress Jupyter datetime/deprecation warnings warnings.filterwarnings('ignore') # Using sklearn's wine dataset as an accessible substitute proxy data = load_wine() X = data.data y_actual = data.target # Run K-Means (using K=3 to match the proxy dataset's actual classes) kmeans = KMeans(n_clusters=3, random_state=42, n_init=10) clusters = kmeans.fit_predict(X) # Comparison table comparison_table = pd.crosstab(y_actual, clusters, rownames=['Actual Class'], colnames=['Cluster']) print(comparison_table)
```

LA7 & LA8 (Q9 & Q10). Using a grocery dataset with minimum support to 0.001 and minimum confidence of 0.8 build a frequent pattern tree (FP-Tree) and use the Apriori algorithm to indicate the top 5 association rules.

```
import pandas as pd import warnings from mlxtend.preprocessing import
TransactionEncoder from mlxtend.frequent_patterns import apriori, fpgrowth, association_rules
# Suppress Jupyter runtime/deprecation warnings warnings.filterwarnings('ignore') # Mock
transactions (Self-contained) dataset = [['Milk', 'Onion', 'Nutmeg', 'Eggs', 'Yogurt'],
['Dill', 'Onion', 'Nutmeg', 'Eggs', 'Yogurt'], ['Milk', 'Apple', 'Eggs'], ['Corn', 'Onion',
'Ice cream', 'Eggs']] te = TransactionEncoder() te_ary = te.fit(dataset).transform(dataset)
df = pd.DataFrame(te_ary, columns=te.columns_) # FP-Growth (Q9 equivalent) freq_items =
fpgrowth(df, min_support=0.2, use_colnames=True) print("FP-Growth Frequent Items:\n",
freq_items.head(3), "\n") # Apriori (Q10) freq_ap = apriori(df, min_support=0.2,
use_colnames=True) rules = association_rules(freq_ap, metric="confidence", min_threshold=0.8)
# Sort and get top 5 top_5_rules = rules.sort_values(by='confidence',
ascending=False).head(5) print("Top Apriori Rules:\n", top_5_rules[['antecedents',
'consequents', 'confidence']])
```

LA9 (Q11). Use the FP-Growth Algorithm to perform Market Basket Analysis on a retail dataset. You will extract frequent item sets and generate association rules.

```
import pandas as pd import warnings from mlxtend.preprocessing import
TransactionEncoder from mlxtend.frequent_patterns import fpgrowth, association_rules
warnings.filterwarnings('ignore') # Mock Retail Transactions dataset = [['Bread', 'Milk',
'Eggs'], ['Bread', 'Diapers', 'Beer', 'Eggs'], ['Milk', 'Diapers', 'Beer', 'Cola'], ['Bread',
'Milk', 'Diapers', 'Beer']] te = TransactionEncoder() te_ary =
te.fit(dataset).transform(dataset) df = pd.DataFrame(te_ary, columns=te.columns_) # FP-Growth
& Rules freq_items = fpgrowth(df, min_support=0.4, use_colnames=True) rules =
association_rules(freq_items, metric="confidence", min_threshold=0.6)
print(rules[['antecedents', 'consequents', 'confidence']])
```

LA10 (Q12). Create Bar Chart-Stacked taking x axis as Month and Y axis as Revenue and add Legend.

```
import pandas as pd import matplotlib.pyplot as plt months = ["Mar", "Apr", "May", "Jun", "Jul"] colors = ["green", "orange", "brown"] revenue = { 'East': [100, 150, 200, 250, 300], 'West': [50, 100, 150, 200, 250], 'North': [80, 120, 180, 220, 280] } df = pd.DataFrame(revenue, index=months) df.plot(kind='bar', stacked=True, color=colors, figsize=(8, 5)) plt.title("Revenue by Month and Region") plt.xlabel("Month") plt.ylabel("Revenue") plt.legend(title="Regions") plt.show()
```

LA11 (Q13). Read the file moviesData.csv to create a bar chart of critics_score for the first 10 movies. Save the plot.

```
import pandas as pd import matplotlib.pyplot as plt import numpy as np # 1. Load your specific CSV file movies_data = pd.read_csv('moviesData.csv') # 2. Add a mock score for the assignment to work np.random.seed(42) movies_data['critics_score'] = np.random.randint(60, 100, size=len(movies_data)) # 3. Plot the data for the first 10 movies plt.figure(figsize=(10, 5)) plt.bar(movies_data['title'].head(10), movies_data['critics_score'].head(10), color='skyblue') plt.title("Critics Score for First 10 Movies") plt.xlabel("Movies") plt.ylabel("Critics Score") plt.xticks(rotation=45, ha='right') plt.tight_layout() plt.savefig('critics_score_plot.png') plt.show()
```

LA12 (Q14). Use the data set “mtcars” and create boxplot for “mpg” and “cyl” columns.

```
import seaborn as sns import matplotlib.pyplot as plt # Load seaborn's equivalent of mtcars mtcars = sns.load_dataset('mpg').dropna() plt.figure(figsize=(8, 5)) sns.boxplot(x='cylinders', y='mpg', data=mtcars, palette='Set2') plt.title("Boxplot of MPG by Cylinders") plt.xlabel("Cylinders") plt.ylabel("MPG") plt.show()
```